

Teoria della dimostrazione, Teoria dei tipi, Proof Assistants e AI

verso una nuova pratica della matematica

Marino Miculan

Scuola Estiva di Logica AILA 2026

Bardonecchia, 31 agosto – 5 settembre 2026

DMIF, Università di Udine | marino.miculan@uniud.it | <https://marino.miculan.org>

LEZIONE 5

Laboratorio Lean II: strutture algebriche elementari e Mathlib

AILA 2026 — Bardonecchia

Obiettivo

Passare da «logica al calcolatore» a **matematica al calcolatore**, e dare un primo assaggio della scala industriale di Mathlib.

Ripasso: gli esercizi di ieri

LE QUATTRO LEGGI DI DE MORGAN — quali valgono costruttivamente?

$$\neg(A \vee B) \rightarrow \neg A \wedge \neg B \quad \checkmark$$

$$\neg A \vee \neg B \rightarrow \neg(A \wedge B) \quad \checkmark$$

$$\neg A \wedge \neg B \rightarrow \neg(A \vee B) \quad \checkmark$$

$$\neg(A \wedge B) \rightarrow \neg A \vee \neg B \quad \times$$

⚠ L'unica che manca non è un caso

Sopra la logica intuizionista è **equivalente** al terzo escluso debole, $\neg A \vee \neg \neg A$: assumerla significa già assumere un frammento di logica classica.

```
example (A B : Prop) :  $\neg(A \wedge B) \rightarrow \neg A \vee \neg B$  := by
  intro h
  by_cases hA : A
    -- qui, e solo qui, entra Classical
    · right; intro hb; exact h ⟨hA, ⟩hb
    · left; exact hA
```

Currying e i due esercizi su [List](#): in `Soluzioni/Foglio2.lean`.

Blocco 1

Formalizzare gruppi e monoidi da zero

Una struttura algebrica è una `class`

```
class MyGroup (G : Type) extends Mul G, One G, Inv G where  
  mul_assoc :  $\forall a\ b\ c : G, (a * b) * c = a * (b * c)$   
  one_mul    :  $\forall a : G, 1 * a = a$   
  mul_one    :  $\forall a : G, a * 1 = a$   
  inv_mul    :  $\forall a : G, a^{-1} * a = 1$ 
```

- ▶ una **class** è una **structure** con *risoluzione automatica delle istanze*:
[MyGroup G] è un argomento che Lean cerca da solo;
- ▶ **extends** realizza l'ereditarietà multipla della gerarchia algebrica;
- ▶ i campi sono *proposizioni*: la struttura porta con sé le sue dimostrazioni. 🔗 L3: Σ -tipi

Il primo teorema: unicità dell'inverso

```
theorem inv_unique {G : Type} [MyGroup G] (a b : G) (h : a * b = 1) :  
  b = a-1 := by  
  calc b = 1 * b           := (MyGroup.one_mul b).symm  
    _ = (a-1 * a) * b      := by rw [MyGroup.inv_mul]  
    _ = a-1 * (a * b)      := MyGroup.mul_assoc _ _ _  
    _ = a-1 * 1           := by rw [h]  
    _ = a-1               := MyGroup.mul_one _
```

Il blocco `calc`

È il modo in cui una catena di uguaglianze «da carta» si scrive in Lean restando leggibile. Ogni passo va giustificato: nessuna scorciatoia, ma il risultato somiglia a una dimostrazione scritta a mano.

Esercizio

1. unicità dell'elemento neutro: se $\forall a. e * a = a$ allora $e = 1$;
2. legge di cancellazione: $a * b = a * c \Rightarrow b = c$;
3. $(a * b)^{-1} = b^{-1} * a^{-1}$;
4. $(a^{-1})^{-1} = a$.

Perché proprio questi

Sono brevi ma **non** si chiudono con **simp** a caso: costringono a ragionare su *quale* assioma serve e in quale direzione riscrivere. È l'analogo formale di un esercizio da primo corso di algebra.

Automazione: che cosa fa davvero

simp	riscrittura (<i>simp set</i>)
ring	anelli commutativi
group	gruppi
omega	aritmetica lineare su $\mathbb{N}/\mathbb{Z}$
decide	predicati decidibili
linarith	disuguaglianze lineari

```
example (x y :  $\mathbb{Z}$ ) : (x + y)^2 = x^2 +  
2*x*y + y^2 := by
```

```
ring
```

```
example (n : Nat) (h : n > 3) : n ≥ 4 :=  
by  
omega
```

```
example : Nat.Prime 97 := by  
decide
```

Attenzione

Tutte producono un **termine di prova** che passa dal kernel  *L1: de Bruijn criterion*. Ma nessuna sostituisce la comprensione degli assiomi: quando falliscono, bisogna sapere perché.

Blocco 2

Mathlib e il «mondo reale»

I nomi di Mathlib si *leggono*

Il nome di un lemma **descrive l'enunciato**, letto da sinistra a destra. Imparata la convenzione, metà della ricerca è fatta.

<code>mul add sub</code>	<code>*</code> <code>+</code> <code>-</code>
<code>neg inv</code>	<code>-a</code> <code>a⁻¹</code>
<code>le lt ne</code>	<code>≤</code> <code><</code> <code>≠</code>
<code>zero one</code>	<code>0</code> <code>1</code>
<code>nonneg pos</code>	<code>0 ≤ a</code> <code>0 < a</code>
<code>_of_</code>	conclusione $\leftarrow$ ipotesi
<code>_iff_</code>	$\leftrightarrow$

```
mul_comm   : a * b = b * a
one_mul    : 1 * a = a
le_of_lt   : a < b → a ≤ b
add_le_add_left :
  b ≤ c → ∀ a, a + b ≤ a + c
sub_nonpos : a - b ≤ 0 ↔ a ≤ b
```

L'apice ' segna una variante dello stesso enunciato. E `#check @mul_comm` mostra sempre la forma completa, con argomenti impliciti e istanze.

Gli strumenti di ricerca

DAL GOAL — tattiche

exact?	chiude il goal, se può
apply?	elenca i lemmi applicabili
rw?	propone riscritture
simp?	mostra che cosa ha usato
hint	prova più tattiche insieme

simp? è il più educativo: sostituisce **simp** con la lista esplicita dei lemmi, e la prova diventa stabile agli aggiornamenti di Mathlib.

Nota

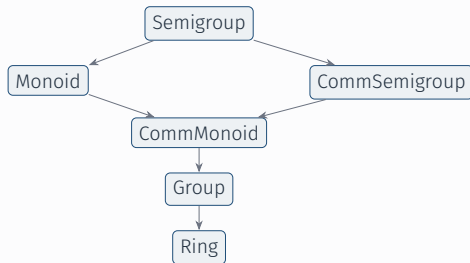
~2M righe, oltre 500 contributori: a questa scala la difficoltà non è dimostrare, è **trovare**.  L6: ed è qui che entra il *premise selection* con l'IA

DALLA FORMA — motori esterni

```
-- Loogle: cerca per forma
|- _ * _ = _ * _
Nat.succ, (_ + _)
"comm"
```

Loogle cerca per *forma* dell'enunciato, per costanti che vi compaiono o per sottostringa del nome; **LeanSearch** accetta l'inglese.

La gerarchia algebrica



Ogni struttura è una **class**; l'ereditarietà è **multipla**, e la risoluzione delle istanze percorre il grafo da sola.

Scrivendo **[Ring R]** si ottengono gratis *tutti* i teoremi su semigrupp, monoidi, gruppi abeliani... centinaia, senza riscrivere nulla.

È il motivo per cui vale la pena formalizzare dentro Mathlib invece che accanto: si eredita, non si ricomincia.

⚠ Ma i cammini si incontrano

A **CommMonoid** si arriva da **Monoid** e da **CommSemigroup**. Che cosa garantisce che le due strade diano la stessa moltiplicazione?

Il diamond problem

Il problema non è che le due istanze siano *diverse*: è che devono essere **definizionalmente** uguali, non solo proposizionalmente. 🔗 L3: *di nuovo* $\equiv$ *contro* *Id*

Se non lo sono, **rw** e **simp** falliscono con errori oscuri — il classico *motive is not type correct* — e due lemmi che parlano dello stesso oggetto non si compongono.

LE DUE CONTROMISURE DI MATHLIB

- ▶ **strutture piatte**: **extends** copia *tutti* i campi del genitore nel figlio, così esiste un solo cammino canonico ai dati;
- ▶ **campi ridondanti**: una struttura porta con sé operazioni che si potrebbero derivare. **Ring** ha un campo **sub**, con l'assioma **sub_eq_add_neg**, invece di definire $a - b := a + (-b)$.

Senza il secondo trucco, la sottrazione *nativa* di $\mathbb{Z}$ non sarebbe definizionalmente quella derivata, e ogni lemma sulla sottrazione andrebbe dimostrato due volte.

PFR

Congettura Polynomial Freiman–Ruzsa. Tao *et al.*, 2023: formalizzata in Lean in ~ 3 settimane, in modo collaborativo e distribuito (*blueprint* + decine di contributori).

PNT forte

Teorema dei numeri primi, versione forte: progetto fermo 18+ mesi, completato in ~ 3 settimane con l'aiuto di un sistema di IA (Math Inc. / «Gauss»), su una sfida di Tao e Kontorovich.

Sphere packing

La dimostrazione di Viazovska (Medaglia Fields) per l'impacchettamento ottimo in dimensione 24, formalizzata in Lean in circa due settimane.

Intuizione

Il collo di bottiglia **non** era trovare la matematica: era tradurla in forma verificabile.

 L6: è lì che l'IA sta avendo impatto

✂ Da portare avanti durante la scuola

1. definire **MyMonoid** e dimostrare l'unicità del neutro;
2. definire un omomorfismo di monoidi e dimostrare che la composizione di omomorfismi è un omomorfismo;
3. un anello elementare: $0 \cdot a = 0$ a partire dagli assiomi;
4. (facoltativo) rifare (1) usando le classi di Mathlib invece delle proprie, e confrontare.

- ▶ J. Avigad e P. Massot. *Mathematics in Lean*. Documentazione online, aggiornata di continuo. 2025. URL:
https://leanprover-community.github.io/mathematics_in_lean/
- ▶ The mathlib Community. «The Lean Mathematical Library». In: *Certified Programs and Proofs (CPP 2020)*. ACM, 2020, pp. 367–381
- ▶ T. Tao, Y. Dillies et al. *A Formalisation of the Polynomial Freiman–Ruzsa Conjecture*. Progetto di formalizzazione collaborativa in Lean 4. 2023. URL:
<https://github.com/teorth/pfr>
- ▶ R. Affeldt et al. «Competing Inheritance Paths in Dependent Type Theory: A Case Study in Functional Analysis». In: *Automated Reasoning (IJCAR 2020)*. Vol. 12167. LNCS. Springer, 2020, pp. 3–20 — sul *diamond problem*.